

Diabetic Retinopathy Classification

CNN Development — Session Handoff Notes

Session date: April 6, 2025

This document summarizes every change made during this session — what was tried, why, what happened, and what the final state of each model is.

1. Infrastructure changes (apply to both models)

Before improving either model, several foundational problems were fixed. These changes should not be reverted.

1.1 Image preprocessing cache

Problem: CropByEye (OpenCV + skimage) ran on every image during training, making data loading catastrophically slow. 4 batches took 57 seconds on a T4 GPU — effectively making training impossible.

Solution: a one-time preprocessing script (`cache_and_dataset.py`) runs CropByEye + Rescale + CenterCrop on all 3500 images and saves them as .npz arrays. Only fast stochastic augmentation runs at training time.

- Cache location: `./cache_224/` (use Google Drive path on Colab to survive session restarts)
- Cache format: float32 numpy arrays, shape (3, 224, 224), one file per image
- Naming: `{split}_{image_id}.npz` — e.g. `train_00042.npz`
- Idempotent: skips already-cached files, safe to interrupt and resume
- Result: 4-batch load time went from 57s → 4.7s (12x speedup)

On Colab: set `CACHE_DIR = '/content/drive/MyDrive/retinopathy_cache_224'` before running the cache script. Without this the cache is lost on every session disconnect.

1.2 CachedRetinopathyDataset

The original RetinopathyDataset was replaced with CachedRetinopathyDataset. Key differences:

- Stores the dataframe as `self.df` (not `self.dataset` — this matters, see section 3.1)
- Loads .npz files directly — no JPEG decoding or OpenCV at training time
- Training augmentation: `RandomHorizontalFlip(0.5)`, `RandomVerticalFlip(0.2)`, `RandomRotation(15)`, `ColorJitter`, `Normalize`
- Evaluation: `Normalize` only — no stochastic transforms

- `num_workers=2` on all DataLoaders (safe on Colab, avoids Windows multiprocessing bug)

1.3 Updated `train_model()` — checkpointing and history

The original function had no resume capability and did not track metrics. The updated version (`train_model_improved.py`) adds:

Checkpointing

- Saves checkpoints/`{run_name}_last.pt` at the end of every epoch
- Contains: model weights, optimizer state, scheduler state, best weights so far, best AUC, early-stopping counter
- Detects the checkpoint on startup and resumes automatically from the next epoch
- Also saves checkpoints/`{run_name}_best.pt` whenever val AUC improves

History tracking

- Returns a tuple: (model, history) — update all call sites accordingly
- history dict contains: `train_loss`, `val_loss`, `train_auc`, `val_auc` — one float per epoch
- Saved to checkpoints/`{run_name}_history.json` alongside the checkpoint
- `plot_training_curves(history, title)` renders loss and AUC side by side with best epoch marked

All `train_model()` calls must be updated to unpack two return values: `model, history = train_model(...)`

3. Model 2 — Fine-tuned network (EfficientNet-B3)

Three training runs were performed, iteratively diagnosing and correcting over/underfitting. All runs used the same `BCEWithLogitsLoss` with `pos_weight` to handle the 73%/27% class imbalance.

3.1 Backbone change: ResNet18 → EfficientNet-B3

The pretrained backbone was upgraded from ResNet18 to EfficientNet-B3:

- EfficientNet-B3: 82.0% ImageNet top-1 accuracy vs 69.8% for ResNet18
- Compound scaling (depth + width + resolution) gives richer, more transferable features
- Parameter count is similar: ~12M vs ~11M — no significant cost increase
- Input: 224x224 (native is 300x300 but 224 works well and keeps the pipeline unchanged)

Bug fix required: the criterion cell originally used `train_dataset.dataset` which does not exist on `CachedRetinopathyDataset`. Change to:

```
train_labels_bin = (train_dataset.df['label'].values > 0).astype(int)
```

3.2 Run 1 — blocks 6, 7, 8 unfrozen

- Unfrozen layers: features[6], features[7], features[8], classifier
- Head: Dropout(0.4) → Linear(1536, 1)
- Optimizer: AdamW — backbone lr=1e-4, head lr=1e-3, weight_decay=1e-4
- Scheduler: CosineAnnealingLR, T_max=25, eta_min=1e-6
- Result: train AUC → 0.89, val AUC plateaus at ~0.73, val loss increases after epoch 7
- Diagnosis: OVERFITTING — too many trainable parameters for 2000 images
- Best val AUC: 0.748 at epoch 7

3.3 Run 2 — block 8 only unfrozen

- Unfrozen layers: features[8], classifier only
- Head: Dropout(0.6) → Linear(1536, 1)
- Optimizer: AdamW — backbone lr=5e-5, head lr=1e-3, weight_decay=1e-3
- Result: train and val AUC nearly identical (~0.71) throughout, both still rising at epoch 25
- Diagnosis: UNDERFITTING — too few trainable parameters, too much regularisation
- Best val AUC: 0.707

3.4 Run 3 — blocks 7 and 8 unfrozen ← current best

- Unfrozen layers: features[7], features[8], classifier
- Head: Dropout(0.5) → Linear(1536, 1)
- Optimizer: AdamW — backbone lr=1e-4, head lr=1e-3, weight_decay=5e-4
- Scheduler: CosineAnnealingLR, T_max=25, eta_min=1e-6
- Result: val AUC converged at ~0.748 from epoch 5, mild overfitting, no divergence
- Best val AUC: 0.7512 at epoch 20 (vs original 0.71 — improvement of +0.041)

Run	Best val AUC	Diagnosis	Action taken
Run 1 — blocks 6,7,8	0.748	Overfit	Froze block 6, raised dropout + wd
Run 2 — block 8 only	0.707	Underfit	Unfroze block 7, reduced dropout + wd
Run 3 — blocks 7,8 (CURRENT)	0.7512	Ok. Early stopping at epoch 19	Best weights saved

4. Model 1 — Custom network (ImprovedCustomNet)

4.1 Problems with the original CustomNet

- Filter progression 6→16→32→64 is far too narrow
- 5x5 kernels throughout — less efficient than two stacked 3x3s
- No skip connections — vanishing gradients prevent learning in deeper layers
- Flat FC head 6400→120→84→1 — many parameters, no spatial invariance
- Result: 0.60 AUC, barely above random

4.2 ResidualBlock — the core building block

A residual block applies two 3x3 convolutions then adds the original input back to the result (the skip connection). This solves vanishing gradients by giving gradients a shortcut path during backpropagation — even if the conv layers cause gradients to shrink, they still flow strongly through the shortcut.

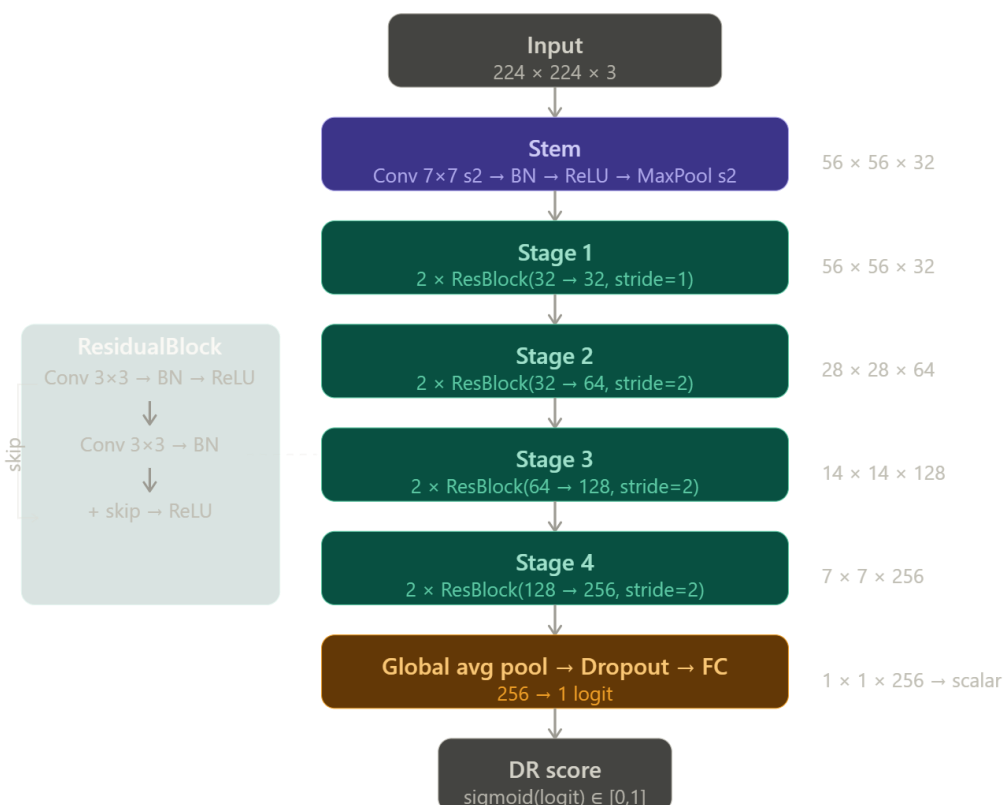
When the number of channels or spatial size changes between input and output, a 1x1 convolution on the skip path (projection shortcut) adjusts the shape so the addition is valid.

- Two Conv2d(3x3) with padding=1, bias=False — preserves spatial size when stride=1
- BatchNorm after each conv — stabilises training, allows higher learning rates
- bias=False on conv: BatchNorm already has a learnable bias (beta), so conv bias is redundant
- ReLU applied after the addition, not before — cleaner gradient signal

4.3 Full architecture

- Stem: Conv(3→32, 7x7, stride=2) + BN + ReLU + MaxPool(stride=2) — 224x224 → 56x56
- Stage 1: 2x ResidualBlock(32→32, stride=1) — 56x56x32
- Stage 2: 2x ResidualBlock(32→64, stride=2) — 28x28x64
- Stage 3: 2x ResidualBlock(64→128, stride=2) — 14x14x128
- Stage 4: 2x ResidualBlock(128→256, stride=2) — 7x7x256
- Head: GlobalAvgPool → Dropout(0.5) → Linear(256→1)
- Total trainable parameters: ~1.7M

Global Average Pooling replaces the flat FC head. It averages each 7x7 feature map to a single value, giving spatial invariance (lesion position does not affect the output magnitude) and reducing the head from Linear(6400,120) to Linear(256,1).



4.4 Weight initialisation

- Conv: Kaiming Normal (He) — preserves activation variance through ReLU layers
- BatchNorm: gamma=1, beta=0 — identity at initialisation
- Linear: Xavier Normal — balanced variance for the final layer

4.5 Training configuration

- Optimizer: AdamW, lr=1e-3, weight_decay=1e-4
- Scheduler: CosineAnnealingLR, T_max=40, eta_min=1e-6
- Epochs: 40, patience: 10 (training from scratch needs more time than fine-tuning)
- Same BCEWithLogitsLoss + pos_weight as Model 2

Training this network clearly showed a learning problem: val AUC was consistently *higher* than train AUC throughout all 27 epochs. To solve this issue the learning rate and dropout were lowered. However, it did not solve the problem.

Therefore, we suspected that the network was too large for the dataset size and reduced it. (New model in the notebook). The training went better but it was still unstable. The next step would be to address this:

“At this point the architecture is fine — the culprit is the **augmentation pipeline**. The cached dataset applies `RandomVerticalFlip` + `RandomRotation(15)` + `ColorJitter` to every

training image. For EfficientNet with 12M pretrained features this is fine — it's already seen every variation. For a 176K network with random initialisation, these transforms make each image so different from the last that the network cannot find any consistent patterns to latch onto. It is essentially trying to learn from a different dataset every epoch. The fix is to give CustomNet a much gentler training transform — horizontal flip only”

5. Results summary

Model	Val AUC	Codabench AUC
Model 1 — ImprovedCustomNet	0.6428	0.55
Model 2 — EfficientNet-B3	0.7512	0.69